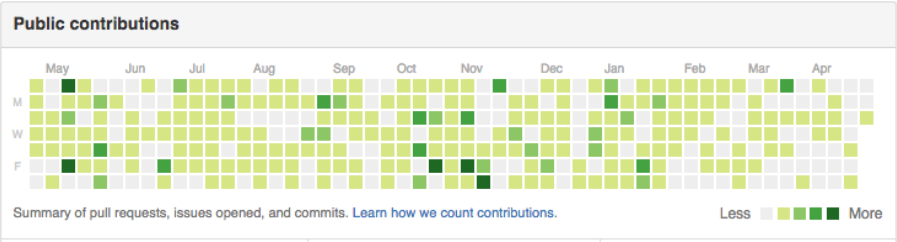
saby

Итоги по строкам и столбцам



за один запрос в PostgreSQL

спикер

Кирилл Боровиков

Компания «Тензор», технический директор

explain.tensor.ru, saby.ru, sbis.ru

https://github.com/Kilor/Presentations

saby

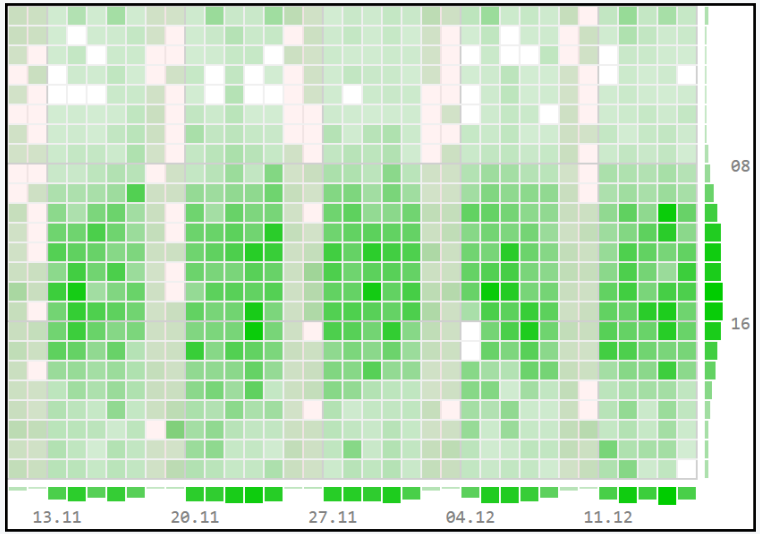
Кирилл Боровиков

- 25+ лет в разработке, почти 20 лет с PostgreSQL
- «Тензор» - сотни проектов, тысячи разработчиков
- технический директор, ведущий архитектор БД
- 150+ хабрапубликаций (@Kilor)
- explain.tensor.ru – анализируем ваши планы

saby

О чем поговорим?

- «тепловые карты» по часам и суткам
- «шахматки» по сотрудникам с итогами
- детализация сводки до «фактов»



Задачи за месяц

	На начало	Получ.	Выполн.	Осталось
	411	166	515	61
Иванов И.В. (Руководитель отдела)	40	40	45	35
Архипцев А.А. (Проект-менеджер)	2	1	1	2
Бирюкова Т.В. (Ведущий дизайнер)	2	11	1	12
Волчихина Т.Е. (Дизайнер (2 категории))	10	5	5	10
Всеволодова А.В. (Проект-менеджер)	30	5	35	0
Петров И.В.	1	2	1	2

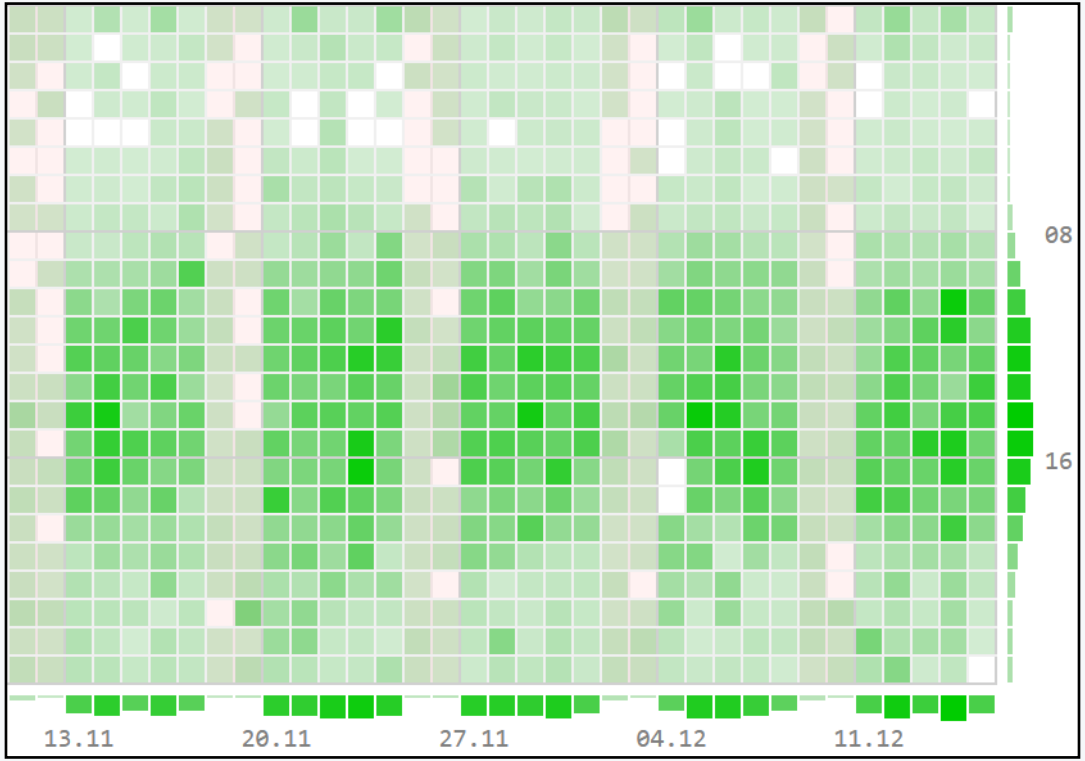
Все	5	52
Мои		7
Моего отдела	5	52
Алексеева О.	2	2
Власова К.	3	21
Золоторев Ф.		20
Константинов Е.		8
Якушева С.		2
Клиенты		
Архпроект Москва	1	5
Аксиома		7
Золотая рыбка	1	2
Золотое дно	2	2

06 дек 12:55	Нетвинское ХКО MSK+4 Требование от налоговой № 4532158 от 6.12.23	19 ч 59 м	Выполнение Подтверждение
11:38	Монтаж-ПРОФИ MSK Пакет документов № 65651 от 6.12.23	20 ч 45 м	Выполнение Утверждение
11:05	Золотое дно MSK Требование от ФСС № 5643211 от 6.12.23	21 ч 12 м	Выполнение Подтверждение
10:01	Русские традиции MSK Больничный: Смирнов Антон Вениаминович	22 ч 16 м	Выполнение Расчет больничного
05 дек 12:55	Лютик MSK+1 Прием на работу: Иванов Константин Игоревич, еще 1	1 д 12 ч	Выполнение
11:38	Палехский завод жестяных изделий MSK Требование о предоставлении документов № 56327 от 5.12.23 пол...	1 д 12 ч	Выполнение Подтверждение
11:05	Золотое дно MSK Добрый день! ИНН компании 7814012472. При добавлении документа не отобр...	1 д 13 ч	Выполнение

saby

Задача: найти количество «фактов» …

- в каждой «клетке» день/час
- в каждом конкретном дне / часе
- на всем интервале



saby

«Размножение» случайных строк

```
CREATE TABLE timefact AS
  SELECT
    '2025-01-01'::date
      + '1 sec'::interval
      * (random() * 365 * 86400)::integer ts -- время факта
  FROM
    generate_series(1, 1e6);
-- без индекса - никуда
CREATE INDEX ON timefact(ts);
```

saby

Находим «координаты» день/час

```
EXPLAIN (ANALYZE, BUFFERS) -- ... давайте-ка посмотрим на план выполнения запроса
SELECT
  ts::date dt                -- день
, extract(hour FROM ts) hr -- час
, count(*)                   -- сколько штук
FROM
  timefact
WHERE
  ts BETWEEN '2025-10-01' AND '2025-10-31'
GROUP BY
  1, 2
ORDER BY
  1, 2;
```

saby

SQL – декларативный язык

некоторые СУБД принимают «подсказки»

demo

Worksheet | Query Builder

```
1  SELECT /* a comment */
2         /*+ full(e) */
3         /*+ full(d) */
4         e.empno,
5         e.ename,
6         d.dname
7    FROM emp e
8    LEFT JOIN dept d
9      ON d.deptno = e.deptno
10  WHERE empno > 7900;
```

PostgreSQL –

но всегда готов рассказать,

как конкретно он планирует

выполнять ваш запрос

(или уже как-то выполнил 😈)

saby

```
Query Text: explain (analyze, buffers, costs off)
SELECT * FROM pg_class WHERE (oid, relname) = (
  SELECT oid, relname FROM pg_class WHERE relkind = 'r' LIMIT 1
);


Index Scan using pg_class_relname_nsp_index on pg_class (actual time=0.049..0.050 rows=1 loops=1)
  Index Cond: (relname = $1)
  Filter: (oid = $0)
  Buffers: shared hit=4
  InitPlan 1 (returns $0,$1)
    ->  Limit (actual time=0.019..0.020 rows=1 loops=1)
          Buffers: shared hit=1
          ->  Seq Scan on pg_class pg_class_1 (actual time=0.015..0.015 rows=1 loops=1)
                Filter: (relkind = 'r'::"char")
                Rows Removed by Filter: 5
                Buffers: shared hit=1
```

saby

saby

explain.tensor.ru

#	node, ms	tree, ms	rows		GroupAggregate / Sort / Index Only Scan using timefact_ts_idx on timefact	sh.ht	tm.rd	tm.wr
		85.027	721		> итоговые результаты (31KB = rows=721 x width=44)	228 1.8MB	192 1.5MB	193 1.5MB
0	17.477	85.027	721 ▼		GroupAggregate (cost=12'208.50..14'274.61 rows=82'626 width=44) (actual time=59.943..85.027 rows=721 loops=1) Group Key: ((ts)::date), (EXTRACT(hour FROM ts)) Buffers: shared hit=228, temp read=192 written=193			
1	37.574	67.550	82'348	DS	-> Sort (cost=12'208.50..12'415.18 rows=82'672 width=36) (actual time=59.895..67.550 rows=82'348 loops=1) Sort Key: ((ts)::date), (EXTRACT(hour FROM ts)) Sort Method: external merge Disk: 1536kB Buffers: shared hit=228, temp read=192 written=193		192 1.5MB	193 1.5MB
2	29.976		82'348		-> Index Only Scan using timefact_ts_idx on timefact (cost=0.42..3'195.22 rows=82'672 width=36) (actual time=0.041..29.976 rows=82'348 loops=1) Index Cond: ((ts >= '2025-10-01 00:00:00'::timestamp without time zone) AND (ts <= '2025-10-31 00:00:00'::timestamp without time zone)) Heap Fetches: 0 Buffers: shared hit=228	228 1.8MB		

на что смотрим: время, RRbF, buffers

Курс «PostgreSQL для начинающих»: #4 – Анализ запросов (ч.1 – как и зачем читать планы)

saby

Убираем сортировку

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  ts::date dt
, extract(hour FROM ts) hr
, count(*)
FROM
  timefact
WHERE
  ts BETWEEN '2025-10-01' AND '2025-10-31'
GROUP BY
  1, 2;
```

saby

Сокращенный вид планов

#	node, ms	tree, ms	rows		GroupAggregate / Sort / Index Only Scan using timefact_ts_idx on timefa	sh.ht	tm.rd	tm.wr
		85.027	721		итоговые результаты (31KB = rows=721 x width=44)	228	192	193
0	17.477	85.027	721		GroupAggregate			
1	37.574	67.550	82'348	DS	-> Sort		192	193
2	29.976		82'348		-> Index Only Scan using timefact_ts_idx on timefact	228		

#	node, ms	tree, ms	rows	HashAggregate / Index Only Scan using timefact_ts_idx on timefact	sh.ht
		61.878	721	итоговые результаты (31KB = rows=721 x width=44)	228
0	28.354	61.878	721	HashAggregate	
1	33.524		82'348	-> Index Only Scan using timefact_ts_idx on timefact	228

PostgreSQL Antipatterns: убираем медленные и ненужные сортировки

saby

Можно просто прочитать

```
SELECT ... -- по "клеткам"
SELECT ... -- по дням
SELECT ... -- по часам
SELECT ... -- итого
```

saby

Неповторимые данные

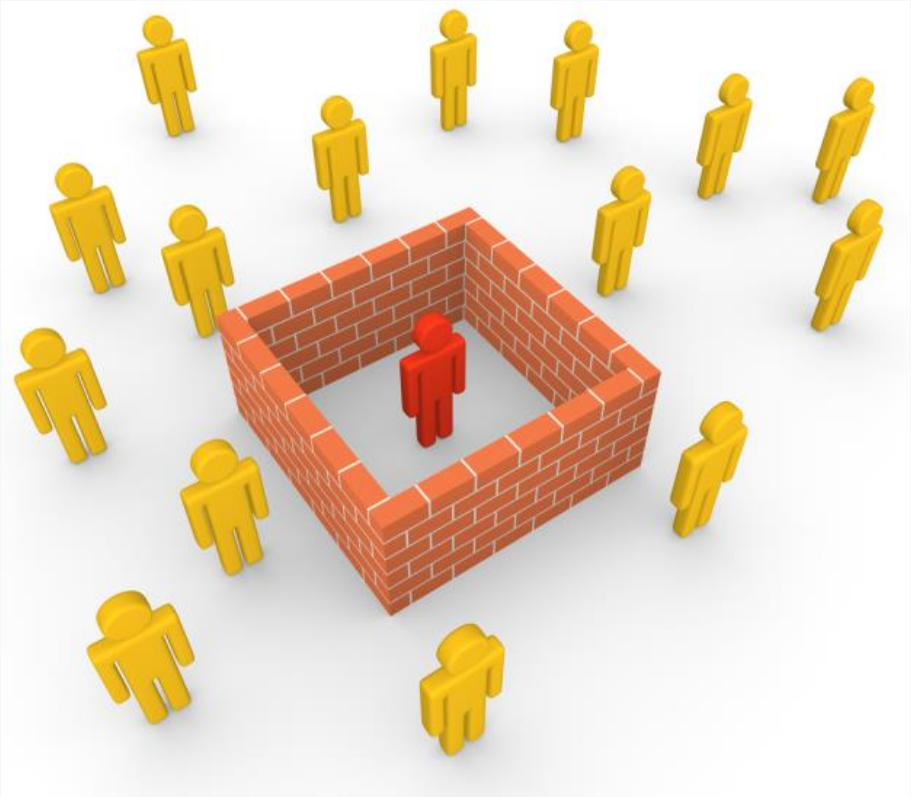
```
SELECT ... -- по "клеткам"
           INSERT ...
SELECT ... -- по дням
           INSERT ...
SELECT ... -- по часам
           INSERT ...
SELECT ... -- итого
```

* вредная параллельная транзакция

saby

Фиксируем данные: изоляция данных

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT ... -- по "клеткам"
SELECT ... -- по дням
SELECT ... -- по часам
SELECT ... -- итого
COMMIT;
```



* прочитали-то все равно все 4 раза

saby

Фиксируем данные: временная таблица

```
CREATE TEMPORARY TABLE preagg AS -- временная таблица
SELECT
  ts::date dt
, extract(hour FROM ts) hr
, count(*)
FROM
  timefact
WHERE
  ts BETWEEN '2025-10-01' AND '2025-10-31'
GROUP BY
  1, 2;
```

saby

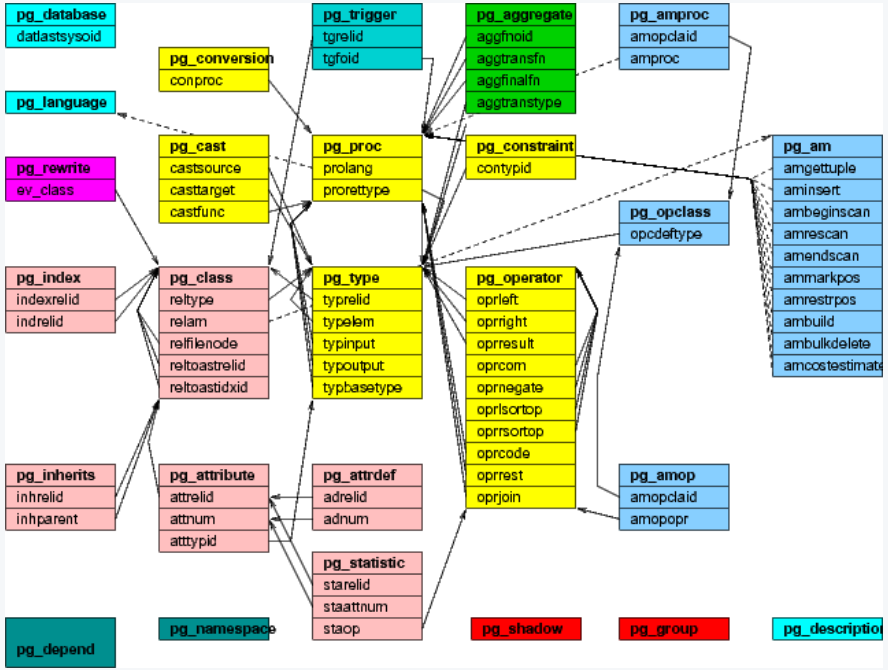
Фиксируем данные: временная таблица

```
-- перечитываем сформированные "клетки"
TABLE preagg;

-- по дням
SELECT
  dt
, sum(count)
FROM
  preagg
GROUP BY dt;
```

```
-- по часам
SELECT
  hr
, sum(count)
FROM
  preagg
GROUP BY hr;

-- итого
SELECT
  sum(count)
FROM
  preagg;
```

saby

Фиксируем данные: временная таблица



* «пухнет» системный каталог (pg_class, pg_attribute, ...)

Олег Бартунов, «Что такое PostgreSQL?», 2005

saby

Фиксируем данные: CTE + UNION ALL

```
WITH preagg AS (
  ...
)
  TABLE preagg -- (dt, hr) = по "клеткам"
UNION ALL
  SELECT       -- (dt, NULL) = по дням
    dt
  , NULL hr
  , sum(count) count
  FROM
    preagg
  GROUP BY dt
...
```

```
  ...
UNION ALL
  SELECT -- (NULL, hr) = по часам
    NULL dt
  , hr
  , sum(count) count
  FROM
    preagg
  GROUP BY hr
UNION ALL
  SELECT -- (NULL, NULL) = итого
    NULL dt
  , NULL hr
  , sum(count) count
  FROM
    preagg;
```

saby

Фиксируем данные: CTE + UNION ALL

#	node, ms	tree, ms	rows		sh.ht
		64.447	777	итоговые результаты (52KB = rows=777 x width=68)	228
0	.062	64.447	777	Append	
1				CTE preagg	
2	28.959	63.120	721	-> HashAggregate	
3	34.161		82'348	-> Index Only Scan using timefact_ts_idx on timefact	228
4	.153	63.474	721	-> Subquery Scan on "*SELECT* 1"	
5	.201	63.321	721	-> CTE Scan on preagg	
6	.209	.260	31	-> HashAggregate	
7	.051		721	-> CTE Scan on preagg preagg_1	
8	.297	.345	24	-> HashAggregate	
9	.048		721	-> CTE Scan on preagg preagg_2	
10	.247	.306	1	-> Aggregate	
11	.059		721	-> CTE Scan on preagg preagg_3	

HashAggregate | Index Only Scan using timefact_ts_idx on timefact

saby

Наводим красоту: GROUPING SETS

```
... GROUP BY dt, hr
UNION ALL
... GROUP BY dt
UNION ALL
... GROUP BY hr
UNION ALL
... GROUP BY ()
```

```
-- то же, но короче
GROUPING SETS (
  (dt, hr)
, (dt    )
, (    hr)
, (      )
)
-- еще короче
GROUPING SETS (
  CUBE(dt, hr)
)
```

saby

Наводим красоту: GROUPING SETS

```
SELECT
  ts::date dt
, extract(hour FROM ts) hr
, count(*)
FROM
  timefact
WHERE
  ts BETWEEN '2025-10-01' AND '2025-10-31'
GROUP BY
  GROUPING SETS (
    CUBE(1, 2)
  );
```

saby

Наводим красоту: GROUPING SETS

#	node, ms	tree, ms	rows		GroupAggregate	Sort	Index Only Scan using timefact_ts_ic	sh.ht	tm.rd	tm.wr
		139.714	777		> итоговые результаты (33KB = rows=777 x width=44)			228 1.8MB	384 3MB	386 3MB
0	72.719	139.714	777 ▼	AT	GroupAggregate (cost=12'208.50..26'386.72 rows=247'879 width=44) (actual time=59.509..139.714 rows=777 loops=1) Group Key: ((ts)::date), (EXTRACT(hour FROM ts)) Group Key: ((ts)::date) Group Key: () Sort Key: (EXTRACT(hour FROM ts)) Group Key: (EXTRACT(hour FROM ts)) Buffers: shared hit=228, temp read=384 written=386				192 1.5MB	193 1.5MB
1	37.036	66.995	82'348	DS	-> Sort (cost=12'208.50..12'415.18 rows=82'672 width=36) (actual time=59.447..66.995 rows=82'348 loops=1) Sort Key: ((ts)::date), (EXTRACT(hour FROM ts)) Sort Method: external merge Disk: 1536kB Buffers: shared hit=228, temp read=192 written=193				192 1.5MB	193 1.5MB
2	29.959		82'348		-> Index Only Scan using timefact_ts_idx on timefact (cost=0.42..3'195.22 rows=82'672 width=36) (actual time=0.044..29.959 rows=82'348 loops=1) Index Cond: ((ts >= '2025-10-01 00:00:00'::timestamp without time zone) AND (ts <= '2025-10-31 00:00:00'::timestamp without time zone)) Heap Fetches: 0 Buffers: shared hit=228			228 1.8MB		

* многократное преобразование значений

saby

Красиво и эффективно: CTE + GROUPING SETS

```
WITH preagg AS (
  SELECT
    ts::date dt
  , extract(hour FROM ts) hr
  , count(*)
  FROM
    timefact
  WHERE
    ts BETWEEN '2025-10-01' AND '2025-10-31'
  GROUP BY
    1, 2
)
...
```

```
...
SELECT
  dt
, hr
, sum(count) count
FROM
  preagg
GROUP BY
  GROUPING SETS (
    CUBE(1, 2)
  );
```

saby

Красиво и эффективно: CTE + GROUPING SETS

```
#  node, ms  tree, ms  rows   Mi HashAggregate | Index Only Scan using timefact_ts_idx on timefact   sh.ht
           62.944     777   > итоговые результаты (52KB = rows=777 x width=68)                        228
                                                                                                      1.8MB
0   1.211  62.944     777 ▼ MixedAggregate  (cost=10'635.70..14'927.66 rows=8'664 width=68) (actual time=62.682..62.944 rows=777 loops=1)
                             Hash Key: ((timefact.ts)::date), (EXTRACT(hour FROM timefact.ts))
                             Hash Key: ((timefact.ts)::date)
                             Hash Key: (EXTRACT(hour FROM timefact.ts))
                             Group Key: ()
                             Batches: 1  Memory Usage: 961kB
                             Buffers: shared hit=228
1  28.317  61.733     721 ▼   -> HashAggregate  (cost=10'635.70..13'166.84 rows=82'626 width=44) (actual time=61.381..61.733 rows=721 loops=1)
                                 Group Key: (timefact.ts)::date, EXTRACT(hour FROM timefact.ts)
                                 Batches: 1  Memory Usage: 1617kB
                                 Buffers: shared hit=228
2  33.416          82'348         -> Index Only Scan using timefact_ts_idx on timefact  (cost=0.42..3'195.22 rows=82'672 width=36) (actual time=0.047..33.416 rows=82'348   228
                                   loops=1)                                                                                                                                  1.8MB
                                       Index Cond: ((ts >= '2025-10-01 00:00:00'::timestamp without time zone) AND (ts <= '2025-10-31 00:00:00'::timestamp without time zone))
                                       Heap Fetches: 0
                                       Buffers: shared hit=228
```

saby

Для настоящих ценителей

```sql
WITH preagg AS (
  SELECT
    ts::date dt
  , extract(hour FROM ts) hr
  , count(*)
  FROM
    timefact
  WHERE
    ts BETWEEN '2025-10-01' AND '2025-10-31'
  GROUP BY
    1, 2
)
...
```

```sql
...
  TABLE preagg -- тут нечего агрегировать
UNION ALL
  SELECT
    dt
  , hr
  , sum(count) count
  FROM
    preagg
  GROUP BY
    GROUPING SETS (
      1, 2, () -- !!!
    );
```

saby

Для настоящих ценителей

#	node, ms	tree, ms	rows	HashAggregate / Index Only Scan using timefact_ts_idx on timefact	sh.ht
		61.888	777	итоговые результаты (52KB = rows=777 x width=68)	228
0	.062	61.888	777	Append	
1				CTE preagg	
2	28.266	60.852	721	-> HashAggregate	
3	32.586		82'348	-> Index Only Scan using timefact_ts_idx on timefact	228
4	.158	61.227	721	-> Subquery Scan on "*SELECT* 1"	
5	.217	61.069	721	-> CTE Scan on preagg	
6	.546	.599	56	-> MixedAggregate	
7	.053		721	-> CTE Scan on preagg preagg_1	

saby

Понять происходящее на сервере PostgreSQL

explain.saby.dev

saby

https://github.com/Kilor/Presentations

Появились вопросы?

Кирилл Боровиков

Компания «Тензор», технический директор

kilor@tensor.ru / explain.tensor.ru / explain.saby.dev